

Proof of conjecture 00000002139: the floor-sum recursion follows the Euclidean algorithm and makes $O(\log m)$ calls

Jackmeson1

2026-10-06

1 The conjecture and its reading

Statement (English, verbatim). Definition: The floor-sum is the summation functional of division with remainder. Conjecture: The complexity of its recursion is logarithmic; and the recursion path is the Euclidean algorithm. (floor-sum Euclidean recursion) (The Chinese text states the same.)

The object. The floor-sum is the function

$$F(n, m, a, b) = \sum_{i=0}^{n-1} \left\lfloor \frac{ai + b}{m} \right\rfloor, \quad n \geq 0, m \geq 1,$$

the sum of the quotients of the divisions with remainder of $ai + b$ by m . This is the function `floor_sum(n, m, a, b)` of the AtCoder Library (ACL), whose documentation says: “It returns $\sum_{i=0}^{n-1} \lfloor \frac{a \times i + b}{m} \rfloor$ ”, with constraints “ $0 \leq n < 2^{32}$ ”, “ $1 \leq m < 2^{32}$ ” and complexity “ $O(\log m)$ ” [ACL-doc]. We first take $a, b \geq 0$; signed a, b are handled in Section 4.

The recursion (a reading choice). The text does not spell out “its recursion”. We take the standard Euclid-like recursion, as implemented in ACL’s `floor_sum_unsigned` [ACL-code]: on modulus m and slope a , add $\binom{n}{2} \lfloor a/m \rfloor + n \lfloor b/m \rfloor$, replace a, b by $a \bmod m, b \bmod m$, put $y = (a \bmod m)n + (b \bmod m)$, and continue with $(n, m, a, b) \leftarrow (\lfloor y/m \rfloor, a \bmod m, m, y \bmod m)$ (“`std::swap(m, a)`”). We formalize two stopping rules. For both we prove that they compute the floor sum and make at most $2\log_2 m + 1$ calls; the exact trace equality, the terminal gcd and the Fibonacci lower bound are proved for `fsRec` only, while for `aclRec` (which may exit early) we prove that its path is a prefix of the Euclid trace:

- `fsRec` stops when the reduced slope $a \bmod m$ is 0. This is the rule of the OI-wiki account of the “Euclid-like algorithm”, which says that (a, c) are repeatedly reduced and swapped “until $a = 0$ ” and that this resembles Euclid’s algorithm on (a, c) [OI-wiki];
- `aclRec` stops when $y < m$. This is exactly the ACL loop (`if (y_max < m) break;`).

The two claims. (i) *The recursion path is the Euclidean algorithm:* the list of (modulus, slope) pairs of the successive calls is the list of argument pairs of Euclid’s algorithm $\gcd(x, y) \rightarrow \gcd(y \bmod x, x) \rightarrow \dots$ started at (m, a) and stopped when the first argument is 0. Its last modulus is $\gcd(m, a)$. For `fsRec` this is an equality of lists. ACL exits early, so for `aclRec` the path is a prefix of the Euclid path. (ii) *The complexity is logarithmic:* each call does a fixed number of

arithmetic operations, so we measure complexity as the number of calls, in the unit-cost model. We prove that it is at most $2\log_2 m + 1$ and at most $2\log_2 \min(a, m) + 2$, whatever n and b are. We also prove that it is at least $\log_2 m + 1$ on an infinite family. So the worst case is $\Theta(\log m)$.

Conventions. $\mathbb{N} \ni 0$. Lean's $/$ and $\%$ on $\mathbb{N}$ are floor division and remainder; we prove that the $\mathbb{N}$ -valued F equals the sum of genuine floors $\lfloor (ai + b)/m \rfloor$ of rationals. $\log_2$ is `Nat.log 2`, the integer part of the binary logarithm. For $m = 0$ the Lean definitions return 0 and the empty path; the theorems below assume $m \geq 1$.

2 Definitions (Lean namespace C2139)

Definition 1. `floorSum` $n\ m\ a\ b = \sum_{i < n} (ai + b)/m$ on $\mathbb{N}$, and `floorSumInt` $n\ m\ a\ b = \sum_{i < n} \lfloor ((ai + b) : \mathbb{Z}) : \mathbb{Q} \rfloor / m$ for $a, b \in \mathbb{Z}$.

Definition 2. `euclidTrace` $x\ y = []$ if $x = 0$, and otherwise $(x, y) :: \text{euclidTrace } (y \bmod x)\ x$. These are the argument pairs of the calls of Euclid's algorithm, with the same recursion as Mathlib's `Nat.gcd_rec`: $\text{gcd}(x, y) = \text{gcd}(y \bmod x, x)$.

Definition 3. `fsRec` and `aclRec` return a pair (value, path), where the path lists the (modulus, slope) pair of every call. With $c = \lfloor n(n-1)/2 \rfloor \cdot \lfloor a/m \rfloor + n \lfloor b/m \rfloor$ and $y = (a \bmod m)n + (b \bmod m)$: `fsRec` $n\ m\ a\ b = (c, [(m, a)])$ if $a \bmod m = 0$, and otherwise $(c + v, (m, a) :: p)$ where $(v, p) = \text{fsRec } \lfloor y/m \rfloor\ (a \bmod m)\ m\ (y \bmod m)$. `aclRec` is the same with the test $y < m$ in place of $a \bmod m = 0$. Both terminate by well-founded recursion on m , since $a \bmod m < m$.

3 Proof

Lemma 4 (`floorSum_reduce`). For $m \geq 1$: $F(n, m, a, b) = \lfloor n(n-1)/2 \rfloor \lfloor a/m \rfloor + n \lfloor b/m \rfloor + F(n, m, a \bmod m, b \bmod m)$.

Proof. $ai + b = ((a \bmod m)i + (b \bmod m)) + m(\lfloor a/m \rfloor i + \lfloor b/m \rfloor)$, so $\lfloor (ai + b)/m \rfloor = \lfloor a/m \rfloor i + \lfloor b/m \rfloor + \lfloor ((a \bmod m)i + b \bmod m)/m \rfloor$. Sum over $i < n$, using $\sum_{i < n} i = \lfloor n(n-1)/2 \rfloor$. $\square$

Lemma 5 (`count_eq`, `floorSum_swap`). If $c \geq 1$, then $\#\{k < N : c(k+1) \leq x\} = \min(N, \lfloor x/c \rfloor)$. If $a \geq 1$ and $b < m$, then with $y = an + b$,

$$F(n, m, a, b) = F(\lfloor y/m \rfloor, a, m, y \bmod m).$$

Proof. The count follows by induction on N , since $c(N+1) \leq x \iff N+1 \leq \lfloor x/c \rfloor$. For the swap, let $N = \lfloor y/m \rfloor$ and $B = y \bmod m$, so $mN + B = an + b$. For $i < n$ we have $\lfloor (ai + b)/m \rfloor \leq N$, so by the count $F(n, m, a, b) = \sum_{i < n} \sum_{k < N} [m(k+1) \leq ai + b]$. For $k < N$ we have $mk + B \leq mN + B - m = an + b - m < an$, so $\lfloor (mk + B)/a \rfloor \leq n$ and $F(N, a, m, B) = \sum_{k < N} \sum_{i < n} [a(i+1) \leq mk + B]$. Exchange the two sums and reflect both indices, $k \mapsto N-1-k$ and $i \mapsto n-1-i$. The two indicators then agree: $m(N-k) \leq a(n-1-i) + b \iff a(i+1) \leq mk + B$, because $m(N-k) + mk = mN$, $a(n-1-i) + a(i+1) = an$ and $mN + B = an + b$. This counts the lattice points under the line by rows and by columns. $\square$

Lemma 6 (`euclidTrace_last`, `euclidTrace_length`, `euclidTrace_length_min`). Let $x \geq 1$. The last pair of `euclidTrace` $x\ y$ has first entry $\text{gcd}(x, y)$. The trace has length at most $2\log_2 x + 1$, and at most $2\log_2 \min(y, x) + 2$.

Proof. For the gcd, use strong induction and $\gcd(x, y) = \gcd(y \bmod x, x)$. If $y \bmod x = 0$, then $\gcd(x, y) = x$. For the first bound, let $r = y \bmod x$. If $r = 0$, the length is 1. Otherwise the trace is $(x, y), (r, x)$ followed by `euclidTrace` s r , where $s = x \bmod r$. Since $r < x$ gives $\lfloor x/r \rfloor \geq 1$, we have $s + r \leq x$ and $s < r$, so $2s < x$. If $s = 0$, the length is $2 \leq 2\log_2 x + 1$, because $x \geq 2$. Otherwise $\log_2 s + 1 = \log_2(2s) \leq \log_2 x$, and induction gives length $\leq 2 + 2\log_2 s + 1 \leq 2\log_2 x + 1$. For the second bound, apply the first one to the tail `euclidTrace` r x , using $r \leq \min(y, x)$. $\square$

Theorem 7 (`fsRec_spec`, `aclRec_spec`). *For all n, a, b and all m : `fsRec` n m a b = $(F(n, m, a, b), \text{euclidTrace } m \ a)$. Also `aclRec` has value $F(n, m, a, b)$, and its path is a prefix of `euclidTrace` m a .*

Proof. Use strong induction on m . The case $m = 0$ is trivial. For $m \geq 1$, apply the reduction lemma. If $a \bmod m = 0$, the remaining sum $F(n, m, 0, b \bmod m)$ is 0, and `euclidTrace` m a = $[(m, a)]$. Otherwise apply the swap with slope $a \bmod m \geq 1$ and offset $b \bmod m < m$, then the induction hypothesis at the modulus $a \bmod m < m$. The tail path is `euclidTrace` $(a \bmod m)$ m , and `euclidTrace` m a = $(m, a) :: \text{that tail}$. For `aclRec`, if $y < m$, every term $\lfloor ((a \bmod m)i + b \bmod m)/m \rfloor$ with $i < n$ is 0, and $[(m, a)]$ is a prefix. If $y \geq m$, then $a \bmod m \neq 0$, and the argument is as before. $\square$

Theorem 8 (`euclidTrace_fib`, `fsRec_fib`). *For every k, n and b , `fsRec` n F_{k+2} F_{k+3} b makes exactly $k + 1$ calls, and $\log_2 F_{k+2} + 1 \leq k + 1$. Here F_j is the Fibonacci number `Nat.fib j`.*

Proof. We have $F_{k+4} \bmod F_{k+3} = F_{k+2}$, because $F_{k+2} < F_{k+3}$. Hence the Euclid trace from (F_{k+3}, F_{k+4}) is one pair longer than the trace from (F_{k+2}, F_{k+3}) . Also $F_{k+2} \leq 2^k$. $\square$

4 Signed a, b

ACL's `floor_sum` for signed a, b first replaces them by their residues mod m [ACL-code]. We prove (`floorSumInt_normalize`):

$$\text{floorSumInt } n \ m \ a \ b = \lfloor a/m \rfloor \lfloor n(n-1)/2 \rfloor + n \lfloor b/m \rfloor + F(n, m, a \bmod m, b \bmod m).$$

Theorem `floorSumInt_recursion` then states that after this one step `fsRec` computes the signed floor-sum, its path is `euclidTrace` m $(a \bmod m)$, and it makes at most $2\log_2 m + 1$ calls.

5 Lean

The main theorem is `C2139.floorSum_recursion_euclid` (assuming $m \geq 1$). It states the conjunction of: F equals the genuine floor-sum (`floorSum_cast`); `fsRec` computes F ; its path equals `euclidTrace` m a ; the last modulus is $\gcd(m, a)$; the number of calls is at most $2\log_2 m + 1$ and at most $2\log_2 \min(a, m) + 2$; `aclRec` computes F , and its path is a prefix of `euclidTrace` m a of length at most $2\log_2 m + 1$. Companion theorems: `C2139.floorSumInt_recursion` (signed) and `C2139.fsRec_fib` (tightness). The file `lean/Conjecture2139/Basic.lean` imports only `Mathlib`. It has no `sorry`, no `native_decide` and no added axioms. `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound` for all three theorems.

Sources

- [ACL-doc] AtCoder Library, math documentation, section `floor_sum`. https://github.com/atcoder/ac-library/blob/master/document_en/math.md
- [ACL-code] AtCoder Library source, `floor_sum_unsigned` in `atcoder/internal_math.hpp` and `floor_sum` in `atcoder/math.hpp`. https://github.com/atcoder/ac-library/blob/master/atcoder/internal_math.hpp
- [OI-wiki] OI-wiki, “Euclid-like algorithm” (in Chinese). <https://github.com/OI-wiki/OI-wiki/blob/master/docs/math/number-theory/euclidean.md>